What you are building

Build exactly this and nothing more:

A firmware failure triage agent for a fake legacy embedded C repo.
Input:

- issue ticket
- fault log
- small repo

Output:

- ranked hypotheses
- most likely root cause
- exact files/functions involved
- minimal patch
- regression risks
- validation tests

That exact shape is already the strongest version of the idea in your PDF.

The core principle

You are **not** asking Claude Code to invent a startup from scratch.

You are asking it to:

1. create a very small controlled repo,
2. create a very small agent app,
3. wire them together,
4. make the demo reliable.

The PDF's strongest advice is to keep scope tight, use one scenario, split ownership hard, and optimize for the live demo rather than architecture.

The one scenario you should lock in immediately

Use this exact scenario from the PDF:

After BMS reconnect, torque commands are received again, but applied torque stays zero until reboot.

Hidden bug:

- `safety_checker.c` sets `torque_inhibit = true` when BMS heartbeat is lost
- `bms_interface.c` detects heartbeat recovery
- but nothing clears `torque_inhibit`
- `torque_controller.c` clamps output torque to zero if inhibit is active

This gives you a believable multi-file bug that is easy for the agent to solve reliably.

The folder you create before doing anything else

Make one project folder, like:

```
None
firmware-triage-agent/
```

Put the uploaded PDF in that folder, renamed to something simple:

```
None
firmware-triage-agent/
  HACKATHON_CONTEXT.pdf
```

Then create one plain-text instruction file for Claude Code:

```
None
firmware-triage-agent/
  CLAUDE_INSTRUCTIONS.md
```

And one scratch/task file both of you can edit:

```
None
firmware-triage-agent/
  WORKPLAN.md
```

The PDF recommends one shared scratchpad with the pitch, file map, bug truth, tasks, and blocked items so both people stay aligned.

What goes into `CLAUDE_INSTRUCTIONS.md`

Paste something like this:

```
None
You are helping us build a hackathon MVP called "Firmware Failure
Triage Agent".

Read HACKATHON_CONTEXT.pdf and follow it closely.

Project goal:
Build a small demo app that investigates a legacy embedded C
codebase using:
- one issue ticket
- one fault log
- a small fake firmware repo

The app must output:
- Summary
- Ranked hypotheses
- Most likely root cause
- Evidence from logs
- Evidence from code
- Minimal patch
- Regression risks
- Validation tests
```

```
Important constraints:
- Keep scope tiny and hackathon-friendly
- No vector DB
- No multi-agent orchestration
- No auth
- No persistent memory
- No real git integration
- No giant repo indexing
- No framework-heavy design

Technical target:
- Python app
- Streamlit UI
- OpenAI-compatible client pointed at NVIDIA NIM
- 3 tools only:
  1. list_repo_files()
  2. read_file(path)
  3. search_repo(query)

Fake firmware repo scenario:
- After BMS reconnect, torque commands are received again, but
applied torque stays zero until reboot
- safety_checker sets torque_inhibit true on BMS timeout
- bms_interface detects heartbeat restore
- torque_controller clamps torque to zero if torque_inhibit is
active
- hidden bug: torque_inhibit is never cleared after reconnect

Required repo structure:
demo_repo/
  main.c
  can_manager.c
  can_manager.h
  bms_interface.c
  bms_interface.h
```

```
  safety_checker.c
  safety_checker.h
  torque_controller.c
  torque_controller.h
  motor_state_machine.c
  motor_state_machine.h
  logs/
    fault_log.txt
  tickets/
    ticket_001.md

Instructions:
- First propose the exact files to create
- Then create the fake embedded repo files
- Then create the Python app files
- Keep all files small and readable
- Add one or two TODOs / legacy comments for realism
- Make the bug span 2-3 files
- Make the system demoable in under 90 seconds
- Prefer reliability over sophistication
```

This matches your PDF almost exactly, including the repo shape, the bug scenario, the output structure, and the “don’t build extra stuff” list.

The exact division of work

Your PDF already gives the right split:

Person A

- fake repo
- planted bug
- ticket
- logs
- Streamlit UI
- wording/story/demo flow

Person B

- Nemotron API call
- tool definitions
- tool loop
- prompt
- output structure
- optional patch generation
- debugging model behavior

Follow that exactly. Do not overlap ownership unless blocked.

How you should actually work with Claude Code

Do **not** both point Claude Code at the entire same folder and let it freestyle.

Instead, use this process.

Step 1: Sit down together for 10 minutes

Before coding, agree on:

- one-sentence pitch
- one demo scenario
- one success condition
- one hard limit
- one fallback

That exact checklist is in the PDF and it's important because it prevents scope drift.

Use this:

- Pitch: "This is a firmware failure triage agent for legacy embedded systems."
- Scenario: "After BMS reconnect, torque stays zero until reboot."
- Success condition: The demo must show root cause, involved files, patch, and tests.
- Hard limit: No vector DB, no multi-agent, no auth, no real git.
- Fallback: If tool calling is flaky, preload the ticket/log and simplify repo search.

Step 2: Make the initial folder and instruction files

One of you creates:

- `HACKATHON_CONTEXT.pdf`
- `CLAUDE_INSTRUCTIONS.md`
- `WORKPLAN.md`

Then both of you read `WORKPLAN.md` and agree on task ownership.

Step 3: Person A and Person B work in parallel

This is crucial. The PDF explicitly says to work in parallel, not serial, and even says Person B should mock tool outputs immediately rather than waiting for the repo to exist.

Person A workflow

Person A's goal is to build the world.

Person A prompt 1

Open Claude Code in the project folder and say:

```
None
Read HACKATHON_CONTEXT.pdf and CLAUDE_INSTRUCTIONS.md.

Your task is ONLY to build the fake firmware world and the demo
UI surface.
Do NOT implement the Nemotron agent loop or tool-calling backend
yet.

Please create:
- demo_repo/ with the exact firmware file structure in the
instructions
- a believable hidden bug where torque_inhibit is set on BMS
timeout and never cleared on heartbeat restore
- one realistic issue ticket in demo_repo/tickets/ticket_001.md
- one realistic fault log in demo_repo/logs/fault_log.txt
- a small Streamlit app shell in app.py that displays the ticket
and log side-by-side and has an "Investigate Failure" button
- placeholder markdown sections for the final report
```

```
Constraints:
- Keep each C file small, around 40-80 lines
- Make the repo feel legacy but readable
- Include one or two TODOs or stale comments
- Keep the bug spread across 2-3 files
- Use searchable terms like heartbeat, torque_inhibit, timeout,
reconnect
- Do not add extra scenarios
- Do not invent more features

Before writing files, show me the file plan.
```

This prompt aligns with the repo plan, the ticket/log artifacts, and Person A's responsibilities from the PDF.

Person A prompt 2

Once the file plan looks good:

```
None
Proceed and create the files now.

After creating them, give me:
1. the exact hidden bug truth in one paragraph
2. the expected correct diagnosis
3. the 3 most relevant files the agent should inspect
4. any wording changes needed to make the ticket and logs more
realistic for a live demo
```

This is important because the PDF says Person A should keep the ground truth and expected diagnosis for testing.

Person A prompt 3

After files exist:

```
None
Now improve app.py only.

Make the Streamlit UI feel polished enough for a hackathon demo:
- title: Firmware Failure Triage Agent
- left column: Issue Ticket
- right column: Fault Log
- button: Investigate Failure
- bottom panel: Agent Report
- add placeholder subsections:
  Summary
  Ranked hypotheses
  Most likely root cause
  Evidence from logs
  Evidence from code
  Minimal patch
  Regression risks
  Validation tests

Do not implement backend logic yet. Keep it clean and simple.
```

That output structure comes directly from the PDF.

Person B workflow

Person B's goal is to build the brain.

Person B prompt 1

Open Claude Code in the same project folder and say:

```
None
Read HACKATHON_CONTEXT.pdf and CLAUDE_INSTRUCTIONS.md.

Your task is ONLY to build the backend agent and tool loop.
```

```
Do NOT modify the fake firmware repo files unless necessary.

Please create:
- tools.py
- agent.py
- prompt.py if useful

Backend requirements:
- Use an OpenAI-compatible client pointed at NVIDIA NIM
- Implement exactly 3 tools:
  1. list_repo_files()
  2. read_file(path)
  3. search_repo(query)
- Use a simple tool-calling loop
- Add a max iteration limit
- Add basic tool-call logging
- Return a structured report with these exact sections:
  Summary
  Ranked hypotheses
  Most likely root cause
  Evidence from logs
  Evidence from code
  Minimal patch
  Regression risks
  Validation tests

Constraints:
- Keep it minimal and hackathon-safe
- No vector DB
- No frameworks
- No multi-agent orchestration
- No persistent memory
- No auth

If demo_repo is not ready yet, first mock the tool outputs with a
tiny hardcoded file list so the loop can be tested in isolation.
```

```
Before writing files, show me the backend plan.
```

This is directly consistent with the PDF's guidance for Person B and the mock-tool shortcut.

Person B prompt 2

After plan approval:

```
None
Proceed and create the backend files now.

Requirements:
- Use the system behavior described in the context PDF
- The agent should investigate the ticket and log, inspect repo
files with tools, form 2-4 ranked hypotheses, identify the most
likely cause, name exact files/functions, propose a minimal
patch, and suggest regression tests
- Keep reasoning concise and evidence-based
- Prefer reliability over sophistication

After creating the files, also print:
1. a sample user prompt to trigger the investigation
2. the expected tool-call sequence for the torque/BMS bug
3. the most likely failure points if the model becomes too
verbose or does not inspect enough files
```

That agent behavior is almost verbatim from the PDF.

Person B prompt 3

Once the backend works:

```
None
Now harden the backend for a hackathon demo.
```

```
Please update the prompt and code so that:
- the model strongly prefers inspecting bms_interface.c,
safety_checker.c, and torque_controller.c when the issue involves
heartbeat restore and torque staying at zero
- the final answer is crisp and formatted for judges
- the patch is short and readable
- the result includes a low/medium/high risk score if easy to add
- tool-call logs are easy to show in the UI later

Do not add any new features outside the current scope.
```

This follows the PDF's advice to constrain the scenario for reliability and optionally add a small wow feature like risk or visible tool use.

How to divide the folder/files

Keep it this simple:

Person A owns

- `app.py`
- `demo_repo/*`
- `demo_repo/tickets/ticket_001.md`
- `demo_repo/logs/fault_log.txt`
- presentation text and UI polish

Person B owns

- `tools.py`
- `agent.py`
- `prompt.py` if used
- model/API config
- tool-call logging

That exact split is recommended in the PDF because it reduces merge conflicts.

How to use Claude Code efficiently instead of chaotically

Rule 1: Ask for file plans before writing

Every big prompt should say:

- “Before writing files, show me the file plan.”

That prevents Claude Code from going off-road.

Rule 2: Keep each prompt single-purpose

Bad:

- “Build the whole hackathon app.”

Good:

- “Build the fake repo.”
- “Build the backend.”
- “Polish the UI.”
- “Harden the system prompt.”

Rule 3: Make Claude edit only owned files

Have each person tell Claude:

- “Only modify files in my ownership scope unless necessary.”

That reduces accidental stomp/merge pain.

Rule 4: Use the PDF as the source of truth

Tell Claude:

- “Follow HACKATHON_CONTEXT.pdf closely.”
- “Do not add features not requested in the PDF.”

Because your PDF already defines the successful MVP and the anti-scope-creep guardrails.

The exact 2-person timeline

First 15 minutes — together

Do this verbally and write it into `WORKPLAN.md`:

- pitch
- scenario
- success criteria
- hard limits
- fallback
- file ownership

The PDF explicitly recommends this first 15-minute alignment step.

Next 45 to 60 minutes — split up

Person A:

- fake repo
- ticket
- log
- UI shell

Person B:

- tools
- agent loop
- prompt
- mock backend if needed

This exact parallel block is also in the PDF.

Next 20 minutes — integrate together

Now merge and wire:

- `app.py` calls `run_agent(...)`
- show report below button
- optionally show tool-call log in sidebar

The PDF says integrate once the core path works, even if ugly. That is one of the most important parts.

Final 20 to 30 minutes — tighten and rehearse

Now do only:

- reduce verbosity
- fix report formatting
- make sure diagnosis is correct
- add fallback screenshot if needed
- rehearse demo speaking parts

That comes straight from the PDF's final-phase advice.

What goes into `WORKPLAN.md`

Use something like:

```
None
# Pitch
Firmware Failure Triage Agent for legacy embedded systems.

# Demo Scenario
After BMS reconnect, torque commands continue, but applied torque
stays zero until reboot.

# Success Criteria
- Agent identifies torque_inhibit latch not cleared
- Names bms_interface.c, safety_checker.c, torque_controller.c
- Proposes a patch
- Lists regression tests

# Hard Limits
- No vector DB
- No multi-agent
- No auth
- No persistent memory
```

```
- No real git integration

# Fallback
If tool loop is flaky:
- preload ticket and log
- keep repo tiny
- simplify search
- constrain prompt harder

# Ownership
Person A:
- demo_repo/*
- app.py
- logs/tickets
- demo polish

Person B:
- agent.py
- tools.py
- prompt.py
- model config
- structured output

# Ground Truth
safety_checker latches torque_inhibit on BMS timeout.
bms_interface sees heartbeat restored but does not clear inhibit.
torque_controller outputs 0 while inhibit remains true.

# Demo Script
A opens.
B explains investigation.
A closes.
```

This uses the exact shared-scratchpad idea from the PDF.

How to sync every 15–20 minutes

Your PDF recommends very short syncs. Use this exact format:

- what's working
- what's blocked
- what gets cut

For example:

- "UI is done."
- "Tool loop works but output is too long."
- "Cut risk score if needed."

Keep it brutally short.

How to test without wasting time

The PDF's 3-level testing model is right. Use it exactly.

Level 1: Tool sanity

Check:

- does `list_repo_files()` work
- does `read_file(path)` work
- does `search_repo("heartbeat")` work

Level 2: Agent sanity

Given ticket + log:

- does the model inspect the right files
- does it mention the real failure path

Level 3: Demo sanity

In the UI:

- can a judge see the problem
- see the investigation

- see the answer
- see the patch/tests

That testing ladder is explicitly in the PDF.

The best fallback if Claude Code starts wandering

If Claude starts overbuilding:
say this:

```
None
Stop adding features.
Return to the MVP defined in HACKATHON_CONTEXT.pdf.

We only need:
- fake embedded repo
- one issue ticket
- one fault log
- 3 tools
- tool-calling loop
- Streamlit button
- structured report
- one patch suggestion

Please remove or ignore anything outside this scope.
```

That MVP checklist is directly from the PDF.

The best fallback if the backend is flaky

If tool calling is inconsistent, reduce scope further:

- keep repo tiny
- harden prompt
- preload ticket/log into prompt
- show files read

- optionally reduce search to substring only

Your PDF explicitly says a constrained “Wizard of Oz” layer is acceptable in a hackathon if it preserves the demo.

The speaking split for the final demo

Your PDF already gives a clean demo role split:

Person A opens
“This is a firmware triage agent for legacy embedded systems.”

Person B explains action
“It reasons across the issue ticket, fault logs, and source files instead of just chatting about code.”

Person A closes
“It reduces codebase archaeology and gives a new engineer a concrete fix path.”

That comes directly from the document.

The biggest thing to avoid

Do **not** let both of you keep prompting Claude to improve the same files.
That is how hackathon velocity dies.

One person owns the world.
One person owns the brain.
Integrate early.
Then stop when the agent reliably says:

- likely root cause: torque inhibit latch never cleared after heartbeat recovery
- involved files: `bms_interface.c`, `safety_checker.c`, `torque_controller.c`
- patch: clear inhibit on reconnect when fault clears
- tests: disconnect/reconnect, repeated reconnect cycles, timeout persistence

That exact stopping condition is essentially the success target described in your PDF.

Use a **split-brain build**:

One person owns the **demo/product surface**.
One person owns the **agent/backend loop**.

That is the fastest way to vibe-code something that still feels coherent.

The main rule

Do **not** both code the same thing at the same time.

Two people on a 2–6 hour hackathon sprint should divide into:

Person A: Product + demo owner

Owns:

- fake repo
- bug ticket
- logs
- Streamlit UI
- wording/story
- final demo flow

Person B: Agent + tool owner

Owns:

- Nemotron API call
- tool definitions
- tool loop
- structured output
- patch generation
- debugging the model behavior

That split minimizes collisions and keeps both people productive the whole time.

The efficient process

Phase 1: Agree on the exact demo in 10 minutes

Before touching code, decide these 5 things:

1. One sentence pitch

Example:
“This is an embedded firmware triage agent that investigates tickets, logs, and legacy C code to find likely root cause and propose a patch.”

2. One demo scenario

Only one.

Example:
“After BMS reconnect, torque stays zero until reboot.”

3. One success condition

Example:
The agent must output:

- likely root cause
- files/functions involved
- patch suggestion
- tests to run

4. One hard limit

Example:
No vector DB, no multi-agent, no auth, no real git integration.

5. One fallback

If the tool loop is flaky, fall back to:

- preloaded file context
- or simplified tool loop
- or even deterministic file reads

This matters a lot. Hackathons are won by demos, not architecture purity.

Phase 2: Split ownership hard

Person A builds the world

Their job is to make the project **look real and understandable**.

They create:

- repo structure
- fake C files
- planted bug
- ticket
- logs
- clean UI
- polished output sections

They should also keep a scratch doc with:

- exact bug truth
- exact file relationships
- exact expected diagnosis

That becomes the ground truth for testing.

Person B builds the brain

Their job is to make the agent **appear intelligent**.

They create:

- `list_repo_files`
- `read_file`
- `search_repo`
- Nemotron call
- tool-calling loop
- prompt
- result formatting

They should also add:

- max iteration limit
- safe fallback message
- logging of tool calls

Phase 3: Work in parallel, not serial

This is the biggest efficiency gain.

While Person A is building the fake repo

Person B should **not wait**.

They can mock tool outputs immediately.

For example, Person B can hardcode temporary returns like:

```python
def list_repo_files():
    return {"files": ["bms_interface.c", "safety_checker.c",
"torque_controller.c"]}
```

That lets them test:

- prompt
- tool schema
- response formatting

Meanwhile Person A builds the actual files.

Then later Person B swaps mock tools for real file reads.

That saves a ton of time.

Best time breakdown for 2 people in a short sprint

First 15 minutes

Together:

- pick scenario
- define success criteria
- assign ownership
- create repo and task list

Next 45 to 60 minutes

Separate:

- Person A: content + UI
- Person B: agent + tools

Next 20 minutes

Together:

- connect backend to UI
- run end-to-end
- fix prompt/formatting issues

Final 20 to 30 minutes

Together:

- demo rehearsal
- tighten output
- simplify anything flaky
- prepare fallback screenshots if needed

The actual working style

Use one shared scratchpad

Could just be a markdown file.

Keep:

- product pitch
- demo script
- file map
- known bug
- tasks
- what is done / blocked

This prevents drift.

Keep one person as final decision-maker

If you both debate too much, you lose time.

For a hackathon, assign:

- one **product decider**
- one **technical decider**

Product decider breaks ties on:

- what to demo
- what to cut
- what wording to use

Technical decider breaks ties on:

- tool loop
- API plumbing
- prompt strategy
- fallback handling

How to vibe-code efficiently without chaos

1. Build ugly first

Do not aim for elegant architecture.

First make it:

- run
- produce output
- demonstrate the core story

Then clean the surface only.

2. Fake aggressively

If something is hard, fake it.

Examples:

- patch generation can be prompt-based only
- risk score can be heuristic
- file search can be plain substring search

- repo can be tiny and synthetic

This is normal hackathon strategy.

3. Lock the scenario early

Do not add a second bug or second workflow unless the first one is done.

One excellent path beats three half-working ones.

4. Optimize for the live demo, not feature count

Ask constantly:
“Will a judge see this in 90 seconds?”

If not, it is probably not worth building.

Suggested file/task split

Person A files

- `app.py`
- `demo_repo/*`
- `tickets/ticket_001.md`
- `logs/fault_log.txt`
- output styling / markdown

Person B files

- `agent.py`
- `tools.py`
- `prompt.py`
- API config
- tool-call logging

That reduces merge conflicts.

How to test efficiently

Do not test everything manually from scratch every time.

Use 3 levels:

Level 1: Tool sanity

Can `list_repo_files`, `read_file`, `search_repo` work?

Level 2: Agent sanity

Given the ticket and log, does the model inspect the correct files?

Level 3: Demo sanity

Does the final screen show:

- the problem
- the investigation
- the answer
- the patch/tests

If all 3 pass, you are demo-ready.

The best collaboration loop

Every 15 to 20 minutes, do a **2-minute sync**:

- what's working
- what's blocked
- what gets cut

Keep it brutally short.

Good sync example:

- "UI is done; I need final report headings."
- "Tool loop works; model is too verbose."
- "Cut patch diff, keep patch summary."

That is enough.

Good division of prompts

Let Person B own the **system prompt**.
Let Person A own the **output shape**.

So Person A says:

- “I need sections called Summary, Root Cause, Patch, Tests.”

Person B changes the prompt to enforce that.

This keeps product and model behavior aligned.

How to avoid the most common 2-person failure mode

The common failure mode is:

- both people keep adding ideas
- nobody locks scope
- integration happens too late
- demo breaks

Prevent that with this rule:

Integrate once the core path works, even if it is ugly.

Do not wait for polish before merging.

Smart shortcut: do a “Wizard of Oz” layer if needed

If the agent is flaky, you can still preserve the demo by constraining it.

Examples:

- preload the ticket and log
- restrict tool set
- reduce repo size
- give the model a highly structured prompt

- manually choose the scenario so the result is reliable

That is still a valid hackathon demo.

A strong workflow for your exact project

Person A

Build:

- fake firmware repo
- one true bug
- one ticket
- one log
- streamlit page with “Investigate Failure” button
- clean result panel

Person B

Build:

- Nemotron client
- function tools
- loop
- strong system prompt
- output structure
- optional patch generation

Together

Refine until the agent says something like:

- likely root cause: torque inhibit latch never cleared after heartbeat recovery
- involved files: `bms_interface.c`, `safety_checker.c`, `torque_controller.c`
- patch: clear inhibit on heartbeat recovery when no active fault remains
- tests: disconnect/reconnect, repeated reconnect cycles, timeout fault persistence

That is the moment you stop building.

What each person should say during the demo

Person A opens

“This is a firmware triage agent for legacy embedded systems.”

Person B explains action

“It reasons across the issue ticket, fault logs, and source files instead of just chatting about code.”

Person A closes

“It reduces codebase archaeology and gives a new engineer a concrete fix path.”

This makes the presentation feel coordinated.

The best repo strategy for 2 people

Have Person A write the fake code in a way that helps Person B succeed.

That means:

- keep bug spread across 2 to 3 files
- include searchable keywords like `heartbeat`, `torque_inhibit`, `timeout`
- avoid too many red herrings
- make logs point toward the right subsystem

Idea 1: Legacy Codebase / Engineering Debug Agent

- An agent that ingests a messy codebase and answers:
    - what this module does
    - where a bug probably lives
    - what files to change
    - how risky a patch is
- Why it could win:
    - Judges love practical dev tools
      Strong fit for LLM reasoning plus retrieval
      Easy wow factor if it can trace through multiple files
- Best version:
    - Focus on one niche: embedded C, robotics, CAN systems, or backend APIs
- How to prove it in the demo
    - Show a task that generic coding agents are not obviously optimized for:
    - Input
        - issue ticket
        - repo
        - log file
        - CAN trace
        - safety rule doc
    - Output
        - ranked hypotheses
        - exact signal path
        - likely file/function
        - why torque is clamped
        - what tests to rerun
        - what safety behavior must remain invariant

■

Nemotron + 3 tools + fake embedded repo + one planted bug + nice report UI

NVIDIA's current docs support tool calling through NIM with `tools` and `tool_choice`, and Nemotron tool calling is supported when detailed thinking is off. NVIDIA also has the NeMo Agent Toolkit, but for a 2-hour hackathon sprint, a plain Python app is the faster path than adopting the full toolkit. NVIDIA's current Nemotron model cards also recommend `temperature=1.0` and `top_p=0.95`. (NVIDIA Docs)

What you are building

A **Firmware Failure Triage Agent**.

User gives:

- issue ticket
- logs
- small fake repo

Agent does:

- picks relevant files
- reads them
- forms hypotheses
- proposes likely root cause
- suggests patch
- lists regression tests

This is the whole product.

The 2-hour build plan

0:00 to 0:10 — Create the fake repo and bug

Do not waste time finding a real repo. Make your own tiny one.

Create this folder structure:

```
None
demo_repo/
  main.c
  can_manager.c
  can_manager.h
  bms_interface.c
  bms_interface.h
  safety_checker.c
  safety_checker.h
  torque_controller.c
  torque_controller.h
  motor_state_machine.c
  motor_state_machine.h
  logs/
    fault_log.txt
  tickets/
    ticket_001.md
```

Use this bug story:

Ticket

> After battery reconnect, torque commands are received again, but applied torque stays zero until reboot.

Actual planted bug

- `safety_checker.c` sets `torque_inhibit = true` when BMS heartbeat is lost
- `bms_interface.c` detects heartbeat recovery
- but nobody clears `torque_inhibit`
- `torque_controller.c` clamps output torque to zero if `torque_inhibit == true`

That gives you a multi-file bug with a believable failure chain.

0:10 to 0:25 — Fill the repo with realistic but small code

Keep each file around 40 to 80 lines.

Use these roles:

- `main.c`: periodic loop
- `can_manager.c`: receives torque commands and heartbeat
- `bms_interface.c`: tracks heartbeat timeout / reconnect
- `safety_checker.c`: fault latch logic
- `torque_controller.c`: applies clamp
- `motor_state_machine.c`: drive/idle/fault states

Add one or two misleading comments and TODOs so it feels legacy.

Example code pattern

```C/C++
// safety_checker.c
#include "safety_checker.h"

static bool torque_inhibit = false;

void safety_set_bms_timeout_fault(void) {
    torque_inhibit = true;
}

bool safety_is_torque_inhibited(void) {
    return torque_inhibit;
}

// TODO: review reconnect handling
```

```C/C++
// bms_interface.c
#include "bms_interface.h"
#include "safety_checker.h"

static bool heartbeat_present = false;

void bms_on_heartbeat_timeout(void) {
    heartbeat_present = false;
    safety_set_bms_timeout_fault();
```

```
}

void bms_on_heartbeat_received(void) {
    heartbeat_present = true;
    // reconnect detected, but fault latch is never cleared
}
```

```C/C++
// torque_controller.c
#include "torque_controller.h"
#include "safety_checker.h"

int torque_compute_applied(int requested_nm) {
    if (safety_is_torque_inhibited()) {
        return 0;
    }
    return requested_nm;
}
```

That is enough.

0:25 to 0:30 — Make the input artifacts

Create `ticket_001.md`:

```None
Title: Torque remains zero after BMS reconnect

Observed:
- During bench testing, disconnecting and reconnecting the BMS
causes torque output to remain at 0 Nm.
- CAN torque commands continue to arrive after reconnect.
- Full system reboot restores torque.
```

```
Expected:
- Torque should resume once BMS heartbeat returns and no active
faults remain.
```

Create `fault_log.txt`:

```
None
[10:21:14.100] INFO torque_cmd_rx requested=42
[10:21:14.120] WARN bms_heartbeat_timeout
[10:21:14.121] WARN safety torque_inhibit=1 reason=BMS_TIMEOUT
[10:21:16.401] INFO bms_heartbeat_restored
[10:21:16.430] INFO torque_cmd_rx requested=38
[10:21:16.431] INFO torque_applied=0
[10:21:16.450] INFO motor_state=DRIVE
```

This log is very strong because it makes the issue obvious enough for the agent to solve, but not too obvious for a human glancing at it.

0:30 to 0:45 — Build the app skeleton

Use:

- Python
- Streamlit
- OpenAI SDK pointed at NVIDIA NIM-compatible endpoint

NVIDIA documents function calling with an OpenAI-compatible format using `tools` and `tool_choice`, which makes this approach straightforward. ([NVIDIA Docs])

Install:

```
Shell
pip install streamlit openai
```

Create:

```
None
app.py
agent.py
tools.py
```

tools.py

You only need 3 tools:

1. `list_repo_files()`
2. `read_file(path)`
3. `search_repo(query)`

That's it.

```python
from pathlib import Path

REPO_ROOT = Path("demo_repo")

def list_repo_files():
    files = []
    for p in REPO_ROOT.rglob("*"):
        if p.is_file():
            files.append(str(p.relative_to(REPO_ROOT)))
    return {"files": sorted(files)}

def read_file(path: str):
    p = REPO_ROOT / path
    if not p.exists() or not p.is_file():
        return {"error": f"File not found: {path}"}
    return {"path": path, "content": p.read_text()}

def search_repo(query: str):
    hits = []
    q = query.lower()
```

```
    for p in REPO_ROOT.rglob("*"):
        if p.is_file():
            text = p.read_text().lower()
            if q in text:
                hits.append(str(p.relative_to(REPO_ROOT)))
    return {"query": query, "matches": hits[:20]}
```

0:45 to 1:05 — Add the Nemotron agent loop

Use a simple tool-calling loop. No framework needed.

agent.py

```python
import json
from openai import OpenAI
from tools import list_repo_files, read_file, search_repo

client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key="YOUR_NVIDIA_API_KEY",
)

MODEL = "nvidia/nemotron-3-super-120b-a12b"

TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "list_repo_files",
            "description": "List all files in the firmware
repository.",
```

```
            "parameters": {"type": "object", "properties": {},
"required": []},
        },
    },
    {
        "type": "function",
        "function": {
            "name": "read_file",
            "description": "Read a file from the firmware
repository.",
            "parameters": {
                "type": "object",
                "properties": {
                    "path": {"type": "string", "description":
"Relative file path"}
                },
                "required": ["path"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "search_repo",
            "description": "Search for a keyword in the
repository.",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description":
"Keyword to search for"}
                },
                "required": ["query"],
            },
        },
    },
```

```
]

TOOL_IMPL = {
    "list_repo_files": lambda **kwargs: list_repo_files(),
    "read_file": lambda **kwargs: read_file(**kwargs),
    "search_repo": lambda **kwargs: search_repo(**kwargs),
}

SYSTEM_PROMPT = """
You are a firmware failure triage agent for a legacy embedded C
codebase.

Your job:
1. Read the bug ticket and logs.
2. Investigate the repository using tools.
3. Form 2-4 ranked hypotheses.
4. Identify the most likely root cause.
5. Name the exact files/functions involved.
6. Propose a minimal patch.
7. Provide regression risks and tests.

Be concrete. Do not say "maybe" unless needed.
Prefer short evidence-backed reasoning.
Return final output with these sections:
- Summary
- Ranked hypotheses
- Most likely root cause
- Evidence
- Suggested patch
- Regression risks
- Tests to run
"""

def run_agent(user_prompt: str):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
```

```
        {"role": "user", "content": user_prompt},
    ]

    for _ in range(8):
        resp = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            tools=TOOLS,
            tool_choice="auto",
            temperature=1.0,
            top_p=0.95,
        )

        msg = resp.choices[0].message
        messages.append(msg)

        if not getattr(msg, "tool_calls", None):
            return msg.content

        for tc in msg.tool_calls:
            name = tc.function.name
            args = json.loads(tc.function.arguments or "{}")
            result = TOOL_IMPL[name](**args)
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": tc.id,
                    "name": name,
                    "content": json.dumps(result),
                }
            )

    return "Agent stopped before final answer."
```

NVIDIA's current docs explicitly describe function calling in this OpenAI-style pattern, including `tools` and `tool_choice`, and note Nemotron tool calling support. The latest Nemotron model card also recommends `temperature=1.0` and `top_p=0.95`. (NVIDIA Docs)

1:05 to 1:20 — Build a dead-simple UI

app.py

```python
import streamlit as st
from pathlib import Path
from agent import run_agent

st.set_page_config(page_title="Firmware Failure Triage Agent",
layout="wide")
st.title("Firmware Failure Triage Agent")

ticket = Path("demo_repo/tickets/ticket_001.md").read_text()
log = Path("demo_repo/logs/fault_log.txt").read_text()

col1, col2 = st.columns(2)

with col1:
    st.subheader("Issue Ticket")
    st.code(ticket, language="markdown")

with col2:
    st.subheader("Fault Log")
    st.code(log, language="text")

if st.button("Investigate Failure"):
    prompt = f"""
Investigate this firmware issue.

Ticket:
{ticket}
```

```
Log:
{log}

Use repository tools to inspect code before answering.
"""
    with st.spinner("Analyzing repository..."):
        result = run_agent(prompt)
    st.subheader("Agent Report")
    st.markdown(result)
```

That is already demoable.

1:20 to 1:35 — Add one wow feature

Add one of these, not all:

Option A: Show the files the agent read

This makes the tool use visible.

You can print tool calls in the UI or log them in a sidebar.

Option B: Add a generated patch

Hardcode a patch template after the diagnosis, or ask the model for a minimal diff.

Example follow-up prompt:

```python
patch_prompt = result + "\n\nNow write a minimal unified diff patch for the fix."
```

Option C: Add a “risk score”

Have the model output:

- Low
- Medium
- High

This feels product-like.

Best choice: **show files read + generate patch**.

1:35 to 1:45 — Make the final output more judge-friendly

Force the model to answer in a crisp structure.

Update the system prompt to require this exact output:

```
None
## Summary
## Ranked hypotheses
## Most likely root cause
## Evidence from logs
## Evidence from code
## Minimal patch
## Regression risks
## Validation tests
```

Judges like organized output.

1:45 to 2:00 — Rehearse the demo

Your demo is only 90 seconds.

Use this script:

Opening

“Generic coding copilots explain code. This agent does firmware failure triage across tickets, logs, and legacy embedded C.”

Show the ticket and log

“Here’s a realistic bench issue: after BMS reconnect, torque stays zero until reboot.”

Run the agent

“As a new engineer, I’d usually spend an hour tracing this. The agent investigates the repo with tools, finds the failure path, and proposes the fix.”

Read the result

Point to:

- ranked hypotheses
- exact files
- root cause
- patch
- tests

Close

“This is not an autocomplete bot. It’s an embedded incident-response agent.”

That framing matters more than the code.

What to say if someone asks “why isn’t this Claude Code?”

Say:

> “Claude Code is general-purpose. This is a domain workflow agent. It reasons across issue tickets, fault logs, firmware modules, and safety constraints to produce a structured triage report and validation plan.”

That is the right answer.

What not to build in these 2 hours

Do not build:

- vector database
- LangGraph
- multi-agent orchestration
- auth
- persistent memory
- real git patch application
- giant repo indexing
- full NeMo Agent Toolkit workflow

NeMo Agent Toolkit is real and useful, but it is broader than what you need for a quick hackathon MVP. For a 2-hour build, the smallest credible demo is better. (NVIDIA Docs)

Your exact MVP checklist

Finish these and stop:

- fake embedded repo with 8 to 10 files
- one believable bug ticket
- one believable log file
- 3 tools: list, read, search
- Nemotron tool-calling loop
- Streamlit button
- structured report
- one patch suggestion

That is enough.

Best final polish

Rename it from “code agent” to one of these:

- **Firmware Failure Triage Agent**
- **Embedded Debug Investigator**
- **Motor Control Incident Copilot**
- **Safety-Aware Firmware Triage**